

Short news(Summarizer) — Fine-tuning Llama-3.2-1B-Instruct with Unsloth (QLoRA)

From dataset → training → GGUF (Ollama) + merged 16-bit (vLLM)

Runtime: Colab → change runtime type → **T4 GPU** (free tier is enough for a 4B model with QLoRA).

This notebook does five things:

1. Install Unsloth and load **Llama-3.2-1B-Instruct** in 8-bit (QLoRA).
2. Attach LoRA adapters (the tiny trainable side-matrices).
3. Load your and apply Llama chat template.
4. Train, watching **train vs. validation loss**.
5. Export to **GGUF for Ollama** and **merged 16-bit for vLLM**.

Mental model recap: the base model stays **frozen**. We only train small LoRA matrices (rank r), scaled by α . QLoRA means the frozen base is quantized to 4-bit so it fits in a T4's ~15 GB.

1. Install

Unsloth pins compatible versions of transformers/trl/peft for you. Pin Unsloth itself if you want reproducibility across class sessions — APIs move fast.

```
# %%capture
!pip install -U unsloth unsloth_zoo
# If a class needs a frozen, reproducible setup, pin instead, e.g.:
# !pip install "unsloth==2026.5.*" "unsloth_zoo==2026.5.*"
```

[Show hidden output](#)

2. Load Llama-3.2-1B-Instruct in 8-bit

- `max_seq_length = 1024`: our records carry a system menu, so they run a little longer than a bare prompt. 1024 is comfortable headroom.
- `load_in_4bit = False`: this is the **Q** in QLoRA — the frozen base is stored in 4-bit, cutting VRAM to ~a quarter.
- `dtype = None`: let Unsloth pick (bfloat16 on newer GPUs, float16 on T4).

```
from unsloth import FastLanguageModel
import torch

max_seq_length = 1024*10
dtype = None          # auto: bf16 where supported, else fp16
load_in_4bit = True   # QLoRA - 4-bit frozen base

model, tokenizer = FastLanguageModel.from_pretrained(
    # model_name      = "unsloth/Qwen3-4B",    # Unsloth's 4-bit-ready Qwen3-4B
    model_name       = "unsloth/Llama-3.2-1B-Instruct",
    max_seq_length   = max_seq_length,
    dtype            = dtype,
    load_in_4bit     = load_in_4bit,
    # full_finetuning = False,                 # default; we're doing LoRA
)
```

```
👉 Unsloth: Will patch your computer to enable 2x faster free finetuning.
👉 Unsloth Zoo will now patch everything to make training faster!
==(====)= Unsloth 2026.8.18: Fast Llama patching. Transformers: 5.5.0.
  \ \ / \  Tesla T4. Num GPUs = 1. Max memory: 14.563 GB. Platform: Linux.
0^0/ \ / \  Torch: 2.11.0+cu128. CUDA: 7.5. CUDA Toolkit: 12.8. Triton: 3.6.0
 \ \ / \ /  Bfloat16 = FALSE. FA [Xformers = 0.0.35. FA2 = False]
"-__-_"    Free license: http://github.com/unslothai/unsloth
Unsloth: Fast downloading is enabled - ignore downloading bars which are red colored!
Loading weights: 100%                               146/146 [00:00<00:00, 1.26it/s]
Unsloth: Will load unsloth/llama-3.2-1b-instruct-unsloth-bnb-4bit as a legacy tokenizer.
```

3. Attach LoRA adapters

This is where the trainable parameters come from. Everything else stays frozen.

- **r (rank) = 16** — capacity of the side-matrices. 8 is fine for pure style; 16 gives a little more room for the persona + "use the provided menu" behaviour. Bigger = more capacity + more params.
- **lora_alpha = 16** — the scaling on the LoRA update. A common rule is $\alpha \approx \text{rank}$ (some use $2 \times \text{rank}$). The **alpha/rank ratio** sets the real strength.
- **target_modules** — which weight matrices get adapters. We hit both attention ((q, k, v, o)) and the feed-forward ($(\text{gate}, \text{up}, \text{down})$) projections — the standard "all-linear" LoRA that gives the best quality.
- **use_gradient_checkpointing = "unsloth"** — trades a little compute for a lot of memory, so longer context fits on a T4.

```
model = FastLanguageModel.get_peft_model(
    model,
    r = 32,
    target_modules = ["q_proj", "k_proj", "v_proj", "o_proj",
                      "gate_proj", "up_proj", "down_proj"],
    lora_alpha = 64,
    lora_dropout = 0,          # 0 is optimized in Unsloth
    bias = "none",            # optimized
    use_gradient_checkpointing = "unsloth",
    random_state = 3407,
    use_rslora = False,       # try True later: scales alpha by 1/sqrt(r)
    loftq_config = None,
)
```

Unsloth 2026.8.18 patched 16 layers with 16 QKV layers, 16 O layers and 16 MLP layers.

4. Bring in your dataset

Upload `news_short_summary_1200_train_val_test` (left sidebar → Files → upload), or pull them from Google Drive / HF. Each line is already in the `messages` format:

```
{ "category": "world",
  "article": "...news...",
  "reference_summary": "...short summary...",
  "difficulty": "medium" }
```

```
# from datasets import load_dataset

# train_dataset = load_dataset("json", data_files="B_e20_train_20.jsonl", split="train")
# val_dataset = load_dataset("json", data_files="A_e20_test_6.jsonl", split="train")

# print(train_dataset)
# print("example:", train_dataset[0]) # a user turn

# from datasets import load_dataset

# # 1. Load the jsonl file
# file_name = "news_short_summary_1200_train_val_test.jsonl"
# dataset = load_dataset("json", data_files=file_name)

# # If your jsonl is a single split, split it into train and validation (e.g., 90/10)
# split_dataset = dataset["train"].train_test_split(test_size=0.30, seed=42)
# train_raw = split_dataset["train"]
# val_raw = split_dataset["test"]

import json

input_files = [
    "news_short_summary_1200_train_val_test.jsonl",
    "out_of_scope_news_summarizer_200.jsonl",
]

output_file = "news_summarizer_v2.jsonl"

with open(output_file, "w", encoding="utf-8") as outfile:
    for file_name in input_files:
        with open(file_name, "r", encoding="utf-8") as infile:
            for line in infile:
                record = json.loads(line)
```

```

        outfile.write(
            json.dumps(record, ensure_ascii=False) + "\n"
        )

print(f"Created: {output_file}")

```

Created: news_summarizer_v2.jsonl

```

from collections import Counter
from datasets import load_dataset

dataset = load_dataset(
    "json",
    data_files="news_summarizer_v2.jsonl"
)

print(
    Counter(dataset["train"]["category"])
)

```

Generating train split: 1400/0 [00:00<00:00, 19881.92 examples/s]

Counter({'out_of_scope': 200, 'sports': 157, 'politics': 157, 'health': 153, 'business': 151, 'world': 150, 'technology': 14

```

dataset = load_dataset(
    "json",
    data_files="news_summarizer_v2.jsonl"
)

split = dataset["train"].train_test_split(
    test_size=0.30,
    seed=42
)

train_raw = split["train"]
temp_raw = split["test"]

val_test = temp_raw.train_test_split(
    test_size=0.50,
    seed=42
)

val_raw = val_test["train"]
test_raw = val_test["test"]

print(len(train_raw))
print(len(val_raw))
print(len(test_raw))

```

980
210
210

```

# train_raw[8]
val_raw[4]

```

```

{'article': 'International health agencies issued updated guidance following the detection of a new infectious disease cluster, with officials emphasizing that the next phase will focus on execution. Officials said surveillance and laboratory testing had been increased while urging authorities to share timely information about new cases. Additional updates are expected as the program progresses. Officials said surveillance and laboratory testing had been increased while urging authorities to share timely information about new cases',
 'reference_summary': 'International health agencies issued updated guidance following the detection of a new infectious disease cluster, with officials emphasizing that the next phase will focus on execution. Officials said surveillance and laboratory testing had been increased while urging authorities to share timely information about new cases. Additional updates are expected as.',
 'category': 'world',
 'difficulty': 'medium'}

```

▼ Apply Qwen3's chat template

Qwen3 uses **ChatML** under the hood (`<|im_start|>` / `<|im_end|>`). You don't write those tokens — you hand Unsloth the clean `messages` list and let `get_chat_template` + `apply_chat_template` wrap each conversation into the exact string Qwen3 was trained on.

`enable_thinking=False` keeps the empty `<think></think>` block out of training targets — for a support-agent persona we want direct replies, not reasoning traces.

```
from unsloth.chat_templates import get_chat_template

# tokenizer = get_chat_template(tokenizer, chat_template="qwen3") # or "qwen-2.5" on older Unsloth
tokenizer = get_chat_template(tokenizer, chat_template="llama-3")

# def format_chat(examples):
#     texts = [
#         tokenizer.apply_chat_template(
#             convo, tokenize=False, add_generation_prompt=False,
#             enable_thinking=False,
#         )
#         for convo in examples["messages"]
#     ]
#     return {"text": texts}

# train_dataset = train_dataset.map(format_chat, batched=True)
# val_dataset = val_dataset.map(format_chat, batched=True)

# print(train_dataset[0]["text"][:600])

# 2. Define a formatting function using chat template
# def format_row(row, tokenizer):
#     # Combine instruction and optional input into the user message
#     user_content = row["instruction"]
#     if row.get("input"):
#         user_content += f"\n\nInput:\n{row['input']}"

#     messages = [
#         {
#             "role": "system",
#             "content": "You are a helpful assistant.",
#         }, # Optional
#         {"role": "user", "content": user_content},
#         {"role": "assistant", "content": row["output"]},
#     ]

#     # Apply tokenizer's chat template
#     text = tokenizer.apply_chat_template(
#         messages, tokenize=False, add_generation_prompt=False
#     )
#     return {"text": text}

# def format_chat(sample, tokenizer):
#     """
#     Formats a news summarization sample into the llama chat template.

#     Expected JSONL fields:
#         article
#         reference_summary
#         category
#         difficulty
#     """

#     # User instruction + article
#     user_content = (
#         "Summarize the following news article in 40-50 words. "
#         "Keep the summary factual, concise, and focused on the most important information. "
#         "Do not add information that is not present in the article.\n\n"
#         f"Category: {sample['category']}\n\n"
#         f"Article:\n{sample['article']}"
#     )

#     # ChatML-style messages
#     messages = [
#         {
#             "role": "user",
#             "content": user_content
```

```

#     },
#     {
#         "role": "assistant",
#         "content": sample["reference_summary"]
#     }
# ]

# # Apply Qwen tokenizer chat template
# text = tokenizer.apply_chat_template(
#     messages,
#     tokenize=False,
#     add_generation_prompt=False
# )

#     return {"text": text}

# # Apply formatting to the training dataset
# train_dataset = train_raw.map(
#     lambda x: format_chat(x, tokenizer),
#     remove_columns=train_raw.column_names,
# )

# val_dataset = val_raw.map(
#     lambda x: format_chat(x, tokenizer),
#     remove_columns=val_raw.column_names,
# )
# print(train_dataset[0]["text"][:600])

def format_chat(sample, tokenizer):

    user_content = (
        "You are a news summarization assistant. "
        "Your only task is to summarize news articles in 40-50 words. "
        "Keep summaries factual, concise, and focused on the most important information. "
        "Do not add information that is not present in the article. "
        "If the input is not a news article, respond exactly with "
        "\"I can only summarize news articles.\\n\\n\""
        f"Article:\\n{sample['article']}")

    messages = [
        {
            "role": "user",
            "content": user_content,
        },
        {
            "role": "assistant",
            "content": sample["reference_summary"],
        },
    ]

    text = tokenizer.apply_chat_template(
        messages,
        tokenize=False,
        add_generation_prompt=False,
    )

    return {"text": text}

train_dataset = train_raw.map(
    lambda x: format_chat(x, tokenizer),
    remove_columns=train_raw.column_names,
)

val_dataset = val_raw.map(
    lambda x: format_chat(x, tokenizer),
    remove_columns=val_raw.column_names,
)

test_dataset = test_raw.map(
    lambda x: format_chat(x, tokenizer),
    remove_columns=test_raw.column_names,
)

```

```
Map: 100% 980/980 [00:00<00:00, 6632.83 examples/s]
Map: 100% 210/210 [00:00<00:00, 3831.01 examples/s]
Map: 100% 210/210 [00:00<00:00, 3938.61 examples/s]
```

```
train_dataset[0]
```

```
{'text': '<|begin_of_text|><|start_header_id|>user<|end_header_id|>\n\nYou are a news summarization assistant. Your only task is to summarize news articles in 40-50 words. Keep summaries factual, concise, and focused on the most important information. Do not add information that is not present in the article. If the input is not a news article, respond exactly with "I can only summarize news articles."\n\nArticle:\nCreate a humorous story about a robot learning to cook.<|eot_id|><|start_header_id|>assistant<|end_header_id|>\n\nI can only summarize news articles.<|eot_id|>'}
```

```
# Crucial for casual LM finetuning
if tokenizer.pad_token is None:
    tokenizer.pad_token = tokenizer.eos_token

# Recommended for modern architectures (Llama, Mistral, Qwen)
tokenizer.padding_side = "right"
```

5. Trainer configuration

Every hyperparameter here maps to something we discussed:

- **learning_rate = 2e-4** — step size per weight update. LoRA's sweet spot. Loss jittering → lower it; barely moving → raise it.
- **per_device_train_batch_size = 2** × **gradient_accumulation_steps = 4** → **effective batch size 8**. The T4 can't hold 8 at once, so we accumulate 4 micro-batches of 2 and apply one combined update — big-batch stability, small-batch memory.
- **num_train_epochs = 3** — with ~352 rows, 2–3 passes is the sweet spot; more risks memorizing.
- **eval_strategy = "steps"** — this is what lets you *watch* train vs. val loss diverge (the overfitting fingerprint).
- **warmup_steps** — ramp the LR up gently so early updates don't thrash the weights.

```
from trl import SFTTrainer, SFTConfig

# trainer = SFTTrainer(
#     model = model,
#     tokenizer = tokenizer,
#     train_dataset = train_dataset,
#     eval_dataset = val_dataset,
#     args = SFTConfig(
#         dataset_text_field = "text",
#         max_seq_length = max_seq_length,
#         per_device_train_batch_size = 2,
#         gradient_accumulation_steps = 4,          # effective batch = 8
#         warmup_steps = 5,
#         num_train_epochs = 3,
#         learning_rate = 2e-5,
#         logging_steps = 5,
#         eval_strategy = "steps",
#         eval_steps = 20,                        # check val loss every 20 steps
#         optim = "adamw_8bit",
#         weight_decay = 0.01,
#         lr_scheduler_type = "linear",
#         seed = 3407,
#         output_dir = "outputs",
#         report_to = "none",
#     ),
# )

trainer = SFTTrainer(
    model=model,
    processing_class=tokenizer,
    train_dataset=train_dataset,
    eval_dataset=val_dataset,

    args=SFTConfig(
        dataset_text_field="text",
        max_seq_length=max_seq_length,

        per_device_train_batch_size=2,
        gradient_accumulation_steps=4,

        warmup_steps=5,
        num_train_epochs=3,
        learning_rate=2e-5,
```

```

        logging_steps=5,
        eval_strategy="steps",
        eval_steps=20,

        optim="adamw_8bit",
        weight_decay=0.01,
        lr_scheduler_type="linear",

        # completion_only_loss=True,

        seed=3407,
        output_dir="outputs",
        report_to="none",
    ),
)

```

Unsloth: We found double BOS tokens - we shall remove one automatically.
 Unsloth: We found double BOS tokens - we shall remove one automatically.

✓ Train only on the assistant's replies

Crucial for instruction tuning: we don't want the model spending capacity learning to *predict the customer's message or the menu*.

`train_on_responses_only` masks the loss so gradients flow **only** from the assistant turns. For Qwen3/ChatML the turn markers are `<|im_start|>user\n` and `<|im_start|>assistant\n`.

```

# from unsloth.chat_templates import train_on_responses_only

# trainer = train_on_responses_only(
#     trainer,
#     instruction_part = "<|im_start|>user\n",
#     response_part    = "<|im_start|>assistant\n",
# )

# Now i switched to llama this step would be skipped

```

```

from unsloth.chat_templates import train_on_responses_only

trainer = train_on_responses_only(
    trainer,
    instruction_part="<|start_header_id|>user<|end_header_id|>",
    response_part="<|start_header_id|>assistant<|end_header_id|>",
)

```

Map: 100% 980/980 [00:00<00:00, 10099.83 examples/s]
 Map: 100% 210/210 [00:00<00:00, 5192.47 examples/s]

```

batch = next(iter(trainer.get_train_dataloader()))

labels = batch["labels"]

print("Batch shape:", labels.shape)

for i in range(labels.shape[0]):
    trainable_tokens = (labels[i] != -100).sum().item()
    print(f"Example {i}: trainable tokens = {trainable_tokens}")

```

Batch shape: torch.Size([2, 228])
 Example 0: trainable tokens = 58
 Example 1: trainable tokens = 52

✓ Go

```

trainer_stats = trainer.train()

```

```

==(====)== Unsloth - 2x faster free finetuning | Num GPUs used = 1
  \ \ / |   Num examples = 840 | Num Epochs = 3 | Total steps = 315
0^0/ \_/ \   Batch size per device = 2 | Gradient accumulation steps = 4
 \ \_/ /     Data Parallel GPUs = 1 | Total batch size (2 x 4 x 1) = 8
  "-__-"    Trainable parameters = 11,272,192 of 1,247,086,592 (0.90% trained)

```

[315/315 09:02, Epoch 3/3]

Step	Training Loss	Validation Loss
------	---------------	-----------------

20	2.090869	1.948525
40	1.285081	1.277853
60	1.176922	1.038388
80	0.980061	0.891943
100	0.820788	0.792347
120	0.799809	0.733111
140	0.622162	0.695699
160	0.677513	0.663907
180	0.702542	0.639668
200	0.588586	0.621757
220	0.541991	0.606714
240	0.501313	0.596624
260	0.594405	0.587797
280	0.676504	0.581060
300	0.659831	0.577435
315	0.460971	0.576553

Unsloth: Restored added_tokens_decoder metadata in outputs/checkpoint-315/tokenizer_config.json.

trainer_stats = trainer.train()

```

==(====)== Unsloth - 2x faster free finetuning | Num GPUs used = 1
  \ \ / |   Num examples = 980 | Num Epochs = 3 | Total steps = 369
0^0/ \_/ \   Batch size per device = 2 | Gradient accumulation steps = 4
 \ \_/ /     Data Parallel GPUs = 1 | Total batch size (2 x 4 x 1) = 8
  "-__-"    Trainable parameters = 22,544,384 of 1,258,358,784 (1.79% trained)

```

`use\_return\_dict` is deprecated! Use `return\_dict` instead!

[369/369 09:24, Epoch 3/3]

Step	Training Loss	Validation Loss
------	---------------	-----------------

20	0.497959	0.471906
40	0.412093	0.331768
60	0.296015	0.225354
80	0.119805	0.176793
100	0.195378	0.160691
120	0.131894	0.152621
140	0.105183	0.141889
160	0.227528	0.141514
180	0.215633	0.134676
200	0.125124	0.133637
220	0.148733	0.139485
240	0.064307	0.134070
260	0.024923	0.133731
280	0.079051	0.131252
300	0.100267	0.131303
320	0.097106	0.130984
340	0.128493	0.130692
360	0.097282	0.129776
369	0.104215	0.129759

Filter: 100%

210/210 [00:00<00:00, 4105.03 examples/s]

Unsloth: Restored added_tokens_decoder metadata in outputs/checkpoint-369/tokenizer_config.json.

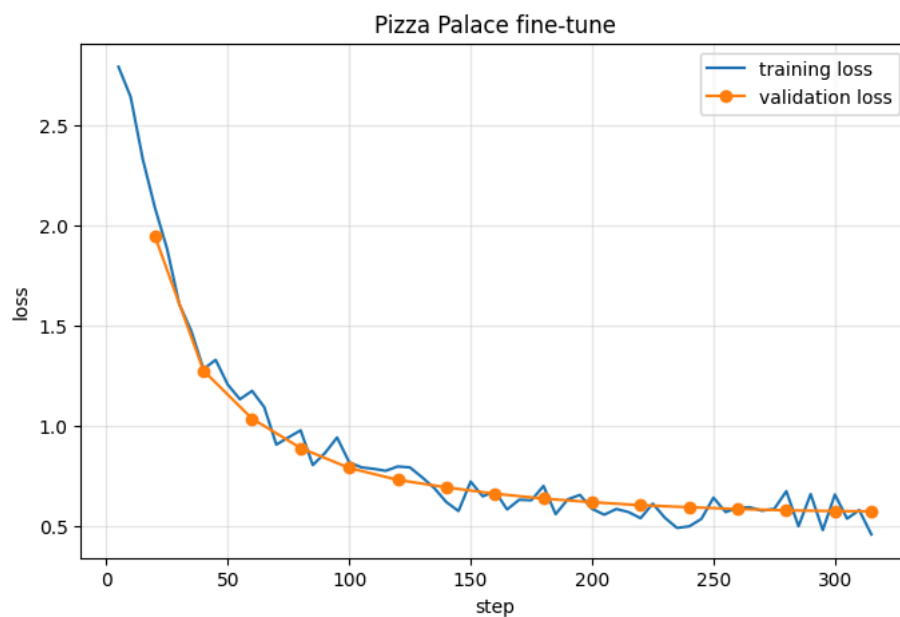
Read the curves

Pull the logged history and plot **training vs. validation loss**. Healthy = both descend and stay close. The moment val flattens/rises while train keeps dropping → overfitting → stop or reduce epochs/rank. Great live demo for the class.

```
import matplotlib.pyplot as plt

hist = trainer.state.log_history
tr = [(h["step"], h["loss"]) for h in hist if "loss" in h]
ev = [(h["step"], h["eval_loss"]) for h in hist if "eval_loss" in h]

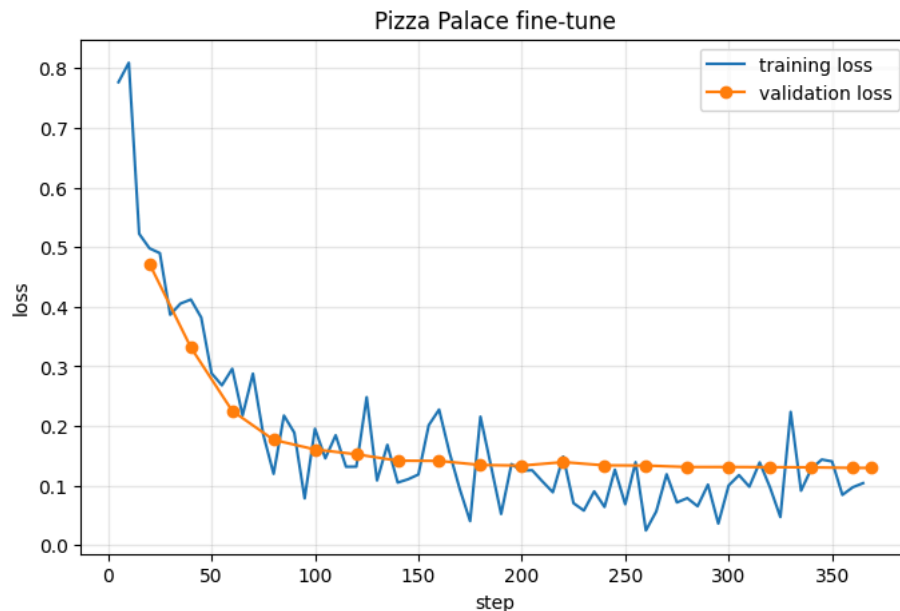
plt.figure(figsize=(8,5))
if tr: plt.plot(*zip(*tr), label="training loss")
if ev: plt.plot(*zip(*ev), label="validation loss", marker="o")
plt.xlabel("step"); plt.ylabel("loss"); plt.legend(); plt.title("Pizza Palace fine-tune"); plt.grid(True, alpha=.3)
plt.show()
```



```
import matplotlib.pyplot as plt

hist = trainer.state.log_history
tr = [(h["step"], h["loss"]) for h in hist if "loss" in h]
ev = [(h["step"], h["eval_loss"]) for h in hist if "eval_loss" in h]

plt.figure(figsize=(8,5))
if tr: plt.plot(*zip(*tr), label="training loss")
if ev: plt.plot(*zip(*ev), label="validation loss", marker="o")
plt.xlabel("step"); plt.ylabel("loss"); plt.legend(); plt.title("Pizza Palace fine-tune"); plt.grid(True, alpha=.3)
plt.show()
```



6. Quick sanity test (before export)

Switch to fast inference and talk to the model. Does it sound like Pizza Palace?

```
# FastLanguageModel.for_inference(model)

# messages = [
#     {"role": "system", "content": "You are a warm, upbeat Pizza Palace support agent. Today's menu:\nMargherita ₹210\nPaner Supreme ₹330\nGarlic Breadsticks ₹120\nCola ₹60\nUse ONLY this menu."},
#     {"role": "user", "content": "hey do u have anything veg"},
# ]
# inputs = tokenizer.apply_chat_template(
#     messages, tokenize=True, add_generation_prompt=True,
#     return_tensors="pt", enable_thinking=False,
# ).to("cuda")

# out = model.generate(input_ids=inputs, max_new_tokens=128, temperature=0.7, top_p=0.9)
# print(tokenizer.decode(out[0], skip_special_tokens=True))
```

The attention mask is not set and cannot be inferred from input because pad token is same as eos token. As a consequence, y Both `max\_new\_tokens` (=128) and `max\_length` (=40960) seem to have been set. `max\_new\_tokens` will take precedence. Please r system

You are a warm, upbeat Pizza Palace support agent. Today's menu:
 Margherita ₹210
 Paner Supreme ₹330
 Garlic Breadsticks ₹120
 Cola ₹60
 Use ONLY this menu.

user
 hey do u have anything veg
 assistant
 <think>

</think>

Hey, great to check! We've got some gems – Margherita (₹210), Paner Supreme (₹330) to start with. Fancy some Garlic Breadst

```
# import re

# # Your sample dataset row
# sample = {
#     'text': (
#         '<|begin_of_text|><|start_header_id|>user<|end_header_id|>\n\n'
#         'How is the ethanol content in E20 measured?<|eot_id|>'
#         '<|start_header_id|>assistant<|end_header_id|>\n\n'
#         'Ethanol content in E20 is measured using laboratory-grade '
#         'analytical methods such as gas chromatography. These tests determine '
#         'the exact percentage of ethanol in the blend. Regulatory agencies '
#         'require these tests to ensure compliance with standards and to verify '
#         'the accuracy of fuel labeling.<|eot_id|>'
#     )
# }

# def parse_to_messages(text_string):
```

```

# # Regex designed exactly for Llama 3 special tokens
# pattern = r"<|start_header_id|>(user|assistant)<|end_header_id|>\n\n(?:<|eot_id|>)"
# matches = re.findall(pattern, text_string, re.DOTALL)

# # Map regex groups directly to standard role/content dictionaries
# return [{"role": role, "content": content.strip()} for role, content in matches]

# # 1. Parse the full sample text
# full_messages = parse_to_messages(sample['text'])

# # 2. Extract only the user prompt for inference input
# inference_input = [msg for msg in full_messages if msg['role'] == 'user']

# # Print results
# print("Parsed Inference Input:")
# print(inference_input)

# inputs = tokenizer.apply_chat_template(
#     inference_input, tokenize=True, add_generation_prompt=True,
#     return_tensors="pt", enable_thinking=False,
#     ).to("cuda")

# out = model.generate(input_ids=inputs, max_new_tokens=128, temperature=0.9, top_p=0.9)
# print(tokenizer.decode(out[0], skip_special_tokens=True))

import re
import torch

# Sample training row from your news summarization dataset
sample = {
    "text": (
        "<|begin_of_text|><|start_header_id|>user<|end_header_id|>\n\n"
        "Summarize the following news article in 40-50 words. "
        "Keep the summary factual, concise, and focused on the most important information. "
        "Do not add information that is not present in the article.\n\n"
        "Category: technology\n\n"
        "Article:\n"
        "A cybersecurity company published a report warning that attacks "
        "targeting small businesses are becoming increasingly automated. "
        "Researchers said attackers are using automated tools to identify "
        "vulnerable systems and urged businesses to strengthen authentication, "
        "regularly update software and maintain secure backups.\n"
        "<|eot_id|>"
        "<|start_header_id|>assistant<|end_header_id|>\n\n"
        "A cybersecurity company warned that attacks on small businesses are "
        "becoming increasingly automated. Researchers said attackers are using "
        "automated tools to identify vulnerable systems and urged businesses "
        "to strengthen authentication, update software regularly and maintain "
        "secure backups to reduce potential damage.\n"
        "<|eot_id|>"
    )
}

def parse_to_messages(text_string):
    """
    Parse Llama 3 formatted training text into
    standard Hugging Face chat messages.
    """
    pattern = (
        r"<|start_header_id|>"
        r"(user|assistant)"
        r"<|end_header_id|>\n\n"
        r"(.*?)(<|eot_id|>)"
    )

    matches = re.findall(
        pattern,
        text_string,
        re.DOTALL
    )

    return [
        {
            "role": role,
            "content": content.strip()
        }
        for role, content in matches
    ]

```

```

# -----
# 1. Parse the complete training example
# -----

full_messages = parse_to_messages(sample["text"])

print("Parsed Messages:")
for message in full_messages:
    print(f"\nROLE: {message['role']}")
    print(f"CONTENT: \n{message['content']}")

# -----
# 2. Keep only the user message for inference
# -----

inference_messages = [
    message
    for message in full_messages
    if message["role"] == "user"
]

print("\n\nInference Input:")
print(inference_messages)

# -----
# 3. Apply Llama 3 chat template for generation
# -----

inputs = tokenizer.apply_chat_template(
    inference_messages,
    tokenize=True,
    add_generation_prompt=True,
    return_tensors="pt",
).to(model.device)

# -----
# 4. Generate the short news summary
# -----

with torch.inference_mode():

    outputs = model.generate(
        input_ids=inputs,
        max_new_tokens=100,
        temperature=0.7,
        top_p=0.9,
        do_sample=True,
    )

# -----
# 5. Decode only the generated response
# -----

generated_tokens = outputs[0][inputs.shape[-1]:]

generated_summary = tokenizer.decode(
    generated_tokens,
    skip_special_tokens=True
).strip()

print("\n\nGenerated News Summary:")
print(generated_summary)

print("\n\nWord Count:", len(generated_summary.split()))

```

Both `max\_new\_tokens` (=100) and `max\_length` (=131072) seem to have been set. `max\_new\_tokens` will take precedence. Please Parsed Messages:

ROLE: user
CONTENT:

Summarize the following news article in 40-50 words. Keep the summary factual, concise, and focused on the most important in

Category: technology

Article:

A cybersecurity company published a report warning that attacks targeting small businesses are becoming increasingly automat

```

ROLE: assistant
CONTENT:
A cybersecurity company warned that attacks on small businesses are becoming increasingly automated. Researchers said attack

Inference Input:
[{'role': 'user', 'content': 'Summarize the following news article in 40-50 words. Keep the summary factual, concise, and fo

Generated News Summary:
A cybersecurity company published a report warning that attacks targeting small businesses are becoming increasingly automat

Word Count: 40

```

```

out = model.generate(input_ids=inputs, max_new_tokens=128, temperature=0.2, top_p=0.9)
print(tokenizer.decode(out[0], skip_special_tokens=True))

```

Both `max\_new\_tokens` (=128) and `max\_length` (=131072) seem to have been set. `max\_new\_tokens` will take precedence. Please user

Summarize the following news article in 40-50 words. Keep the summary factual, concise, and focused on the most important in

Category: technology

Article:

A cybersecurity company published a report warning that attacks targeting small businesses are becoming increasingly automat

A cybersecurity company published a report warning that attacks targeting small businesses are becoming increasingly automat

```

val_dataset[4]

```

```

{'text': '<|begin_of_text|><|start_header_id|>user<|end_header_id|>\n\nSummarize the following news article in 40-50 words.
Keep the summary factual, concise, and focused on the most important information. Do not add information that is not
present in the article.\n\nCategory: world\n\nArticle:\nInternational health agencies issued updated guidance following the
detection of a new infectious disease cluster, with officials emphasizing that the next phase will focus on execution.
Officials said surveillance and laboratory testing had been increased while urging authorities to share timely information
about new cases. Additional updates are expected as the program progresses. Officials said surveillance and laboratory
testing had been increased while urging authorities to share timely information about new cases<|eot_id|>
<|start_header_id|>assistant<|end_header_id|>\n\nInternational health agencies issued updated guidance following the
detection of a new infectious disease cluster, with officials emphasizing that the next phase will focus on execution.
Officials said surveillance and laboratory testing had been increased while urging authorities to share timely information
about new cases. Additional updates are expected as.<|eot_id|>'}

```

```

# FastLanguageModel.for_inference(model)

# messages = [
#     {"role": "system", "content": "You are a warm, upbeat Pizza Palace support agent. Today's menu:\nMargherita ₹210\nPar
#     {"role": "user", "content": "Any drinks are available?"},
# ]
# inputs = tokenizer.apply_chat_template(
#     messages, tokenize=True, add_generation_prompt=True,
#     return_tensors="pt", enable_thinking=False,
# ).to("cuda")

# out = model.generate(input_ids=inputs, max_new_tokens=128, temperature=0.7, top_p=0.9)
# print(tokenizer.decode(out[0], skip_special_tokens=True))

```

```

import torch

# Put model in inference mode
model.eval()

# Test news article
test_article = """
The central bank announced that it will keep its benchmark interest rate
unchanged amid persistent inflation concerns and uncertain global economic
conditions. Officials said domestic economic activity remains resilient,
while food prices and international financial developments will continue
to influence future policy decisions.
"""

prompt = (
    "You are a news summarization assistant. "
    "Your only task is to summarize news articles in 40-50 words. "
    "Keep summaries factual, concise, and focused on the most important information. "
    "Do not add information that is not present in the article. "
    "If the input is not a news article, respond exactly with "
    "\"I can only summarize news articles.\"\n\n"
    f"Article:\n{test_article}"
)

```

```

messages = [
    {
        "role": "user",
        "content": prompt
    }
]

inputs = tokenizer.apply_chat_template(
    messages,
    tokenize=True,
    add_generation_prompt=True,
    return_tensors="pt",
).to(model.device)

with torch.inference_mode():
    outputs = model.generate(
        input_ids=inputs,
        max_new_tokens=100,
        temperature=0.7,
        top_p=0.9,
        do_sample=True,
    )

# Decode only newly generated tokens
generated_tokens = outputs[0][inputs.shape[-1]:]

response = tokenizer.decode(
    generated_tokens,
    skip_special_tokens=True
).strip()

print("Generated Summary:")
print(response)
print("\nWord Count:", len(response.split()))

```

The attention mask is not set and cannot be inferred from input because pad token is same as eos token. As a consequence, yc
Both `max\_new\_tokens` (=100) and `max\_length` (=131072) seem to have been set. `max\_new\_tokens` will take precedence. Please
Generated Summary:

The central bank announced that it will keep its benchmark interest rate unchanged amid persistent inflation concerns and uncertain global economic conditions. Officials said domestic economic activity remains resilient, while food prices and international financial developments will continue to influence future policy decisions.

Word Count: 43

```

test_question = "What is 125 multiplied by 24?"

messages = [
    {
        "role": "user",
        "content": (
            "You are a news summarization assistant. "
            "Your only task is to summarize news articles in 40-50 words. "
            "If the input is not a news article, respond exactly with "
            "\"I can only summarize news articles.\\n\\n\\n"
            f"Article:\\n{test_question}"
        )
    }
]

inputs = tokenizer.apply_chat_template(
    messages,
    tokenize=True,
    add_generation_prompt=True,
    return_tensors="pt",
).to(model.device)

with torch.inference_mode():
    outputs = model.generate(
        input_ids=inputs,
        max_new_tokens=30,
        do_sample=False,
    )

generated_tokens = outputs[0][inputs.shape[-1]:]

response = tokenizer.decode(
    generated_tokens,
    skip_special_tokens=True
).strip()

```

```
print("\nOut-of-Scope Test:")
print(response)
```

Both `max\_new\_tokens` (=30) and `max\_length` (=131072) seem to have been set. `max\_new\_tokens` will take precedence. Please r

Out-of-Scope Test:
I can only summarize news articles.

```
import re
import torch

model.eval()

def parse_sample(text):
    pattern = (
        r"<\\|start_header_id\\|>"
        r"(user|assistant)"
        r"<\\|end_header_id\\|>\\n\\n"
        r"(.*)?(?=<\\|eot_id\\|>)"
    )

    matches = re.findall(pattern, text, re.DOTALL)

    messages = [
        {
            "role": role,
            "content": content.strip()
        }
        for role, content in matches
    ]

    return messages

def generate_response(user_content):
    messages = [
        {
            "role": "user",
            "content": user_content
        }
    ]

    inputs = tokenizer.apply_chat_template(
        messages,
        tokenize=True,
        add_generation_prompt=True,
        return_tensors="pt",
    ).to(model.device)

    with torch.inference_mode():
        outputs = model.generate(
            input_ids=inputs,
            max_new_tokens=100,
            do_sample=False,
        )

    generated_tokens = outputs[0][inputs.shape[-1]:]

    return tokenizer.decode(
        generated_tokens,
        skip_special_tokens=True
    ).strip()

# -----
# Test samples from test_dataset
# -----

for i in range(min(10, len(test_dataset))):

    sample = test_dataset[i]

    # Parse formatted Llama text
    messages = parse_sample(sample["text"])

    user_message = next(
        m["content"] for m in messages
        if m["role"] == "user"
    )
```

```

expected_response = next(
    m["content"] for m in messages
    if m["role"] == "assistant"
)

# Generate model response
generated_response = generate_response(user_message)

print("=" * 100)
print(f"TEST {i + 1}")

print("\nUSER INPUT:")
print(user_message)

print("\nEXPECTED:")
print(expected_response)

print("\nMODEL OUTPUT:")
print(generated_response)

print("\nWORD COUNT:")
print(len(generated_response.split()))

```

Both `max\_new\_tokens` (=100) and `max\_length` (=131072) seem to have been set. `max\_new\_tokens` will take precedence. Please
Both `max\_new\_tokens` (=100) and `max\_length` (=131072) seem to have been set. `max\_new\_tokens` will take precedence. Please

=====

TEST 1

USER INPUT:

You are a news summarization assistant. Your only task is to summarize news articles in 40-50 words. Keep summaries factual

Article:

How do I convert a JSON string into a Python dictionary?

EXPECTED:

I can only summarize news articles.

MODEL OUTPUT:

I can only summarize news articles.

WORD COUNT:

6

Both `max\_new\_tokens` (=100) and `max\_length` (=131072) seem to have been set. `max\_new\_tokens` will take precedence. Please
=====

TEST 2

USER INPUT:

You are a news summarization assistant. Your only task is to summarize news articles in 40-50 words. Keep summaries factual

Article:

An automaker unveiled an investment plan for expanding electric vehicle manufacturing. The program includes new assembly e

EXPECTED:

An automaker unveiled an investment plan for expanding electric vehicle manufacturing. The program includes new assembly e

MODEL OUTPUT:

An automaker unveiled an investment plan for expanding electric vehicle manufacturing. The program includes new assembly e

WORD COUNT:

47

Both `max\_new\_tokens` (=100) and `max\_length` (=131072) seem to have been set. `max\_new\_tokens` will take precedence. Please
=====

TEST 3

USER INPUT:

You are a news summarization assistant. Your only task is to summarize news articles in 40-50 words. Keep summaries factual

Article:

The national cricket team won a closely contested international match. The side recovered from an early setback and secure

EXPECTED:

The national cricket team won a closely contested international match. The side recovered from an early setback and secure

MODEL OUTPUT:

The national cricket team won a closely contested international match. The side recovered from an early setback and secure

WORD COUNT:

41

Both `max\_new\_tokens` (=100) and `max\_length` (=131072) seem to have been set. `max\_new\_tokens` will take precedence. Please
=====

TEST 4

test_dataset[0]

```
{
  'text': '<|begin_of_text|><|start_header_id|>user<|end_header_id|>\n\nYou are a news summarization assistant. Your only task is to summarize news articles in 40-50 words. Keep summaries factual, concise, and focused on the most important information. Do not add information that is not present in the article. If the input is not a news article, respond exactly
```



```
with "I can only summarize news articles."
Article:
How do I convert a JSON string into a Python dictionary?
<|start_header_id|>assistant<|end_header_id|>
I can only summarize news articles.<|eot_id|>}
```

7. Save the LoRA adapters (lightweight checkpoint)

Just the adapters — a few MB. Useful to keep, and to re-merge later without retraining.

```
model.save_pretrained("short_news_lora_v2")      # local
tokenizer.save_pretrained("short_news_lora_v2")
# model.push_to_hub("your-username/pizza-qwen3-lora", token="hf_...") # optional
```

```
Unsloth: Restored added_tokens_decoder metadata in short_news_lora_v2/tokenizer_config.json.
('short_news_lora_v2/tokenizer_config.json',
 'short_news_lora_v2/chat_template.jinja',
 'short_news_lora_v2/tokenizer.json')
```

8. Export → GGUF for Ollama

`save_pretrained_gguf` merges the LoRA into the base, converts to GGUF via llama.cpp, and quantizes. Pick a quant:

- **q4_k_m** — best size/quality trade-off, the usual choice for laptops/edge.
- **q8_0** — larger, higher fidelity.
- **f16** — full precision GGUF (biggest).

This step compiles llama.cpp the first time, so give it a few minutes.

```
model.save_pretrained_gguf(
    "short_news_gguf_v2",
    tokenizer,
    quantization_method = "q4_k_m",  # try "q8_0" for higher fidelity
)
# Produces e.g. short_news_gguf/unsloth.Q4_K_M.gguf
!ls -lh short_news_gguf_v2
```

```

Unsloth: ##### The current model auto adds a BOS token.
Unsloth: ##### Your chat template has a BOS token. We shall remove it temporarily.
Unsloth: Merging model weights to 16-bit format...
config.json: 100%                               894/894 [00:00<00:00, 88.8kB/s]
Unsloth: Restored added_tokens_decoder metadata in short_news_gguf_v2/tokenizer_config.json.
Found HuggingFace hub cache directory: /root/.cache/huggingface/hub
Checking cache directory for required files...
Cache check failed: model.safetensors not found in local cache.
Not all required files found in cache. Will proceed with downloading.
Checking cache directory for required files...
Cache check failed: tokenizer.model not found in local cache.
Not all required files found in cache. Will proceed with downloading.
Unsloth: Preparing safetensor model files:   0%|          | 0/1 [00:00<?, ?it/s]

model.safetensors: reconstructing file: 100%                               2.47GB / 2.47GB, 110MB/s

model.safetensors: downloading bytes:                               2.14GB, 113MB/s

Unsloth: Preparing safetensor model files: 100%|██████████| 1/1 [00:19<00:00, 19.20s/it]
Note: tokenizer.model not found (this is OK for non-SentencePiece models)
Unsloth: Merging weights into 16bit: 100%|██████████| 1/1 [00:53<00:00, 53.63s/it]
Unsloth: Merge process complete. Saved to `/content/short_news_gguf_v2`
Unsloth: Converting to GGUF format...
==(====)= Unsloth: Conversion from HF to GGUF information
  \ \ / \ [0] Installing llama.cpp might take 3 minutes.
0^0/ \_/ \ [1] Converting HF to GGUF f16 might take 3 minutes.
 \ \ / / [2] Converting GGUF f16 to ['q4_k_m'] might take 10 minutes each.
  "-__-" In total, you will have to wait at least 16 minutes.

Unsloth: Installing llama.cpp. This might take 3 minutes...
Unsloth: Installing prebuilt llama.cpp b10472-mix-4b653db (app-b10472-mix-4b653db-linux-x64-cpu.tar.gz) - skipping compilation
Unsloth: Preparing converter script...
Unsloth: [1] Converting model into f16 GGUF format.
This might take 3 minutes...
Unsloth: Initial conversion completed! Files: ['short_news_gguf_v2_gguf/llama-3.2-1b-instruct.F16.gguf']
Unsloth: [2] Converting GGUF f16 into q4_k_m. This might take 10 minutes...
Unsloth: ##### The current model auto adds a BOS token.
Unsloth: ##### We removed it in GGUF's chat template for you.
Unsloth: Model files cleanup...
Unsloth: All GGUF conversions completed successfully!
Generated files: ['short_news_gguf_v2_gguf/llama-3.2-1b-instruct.Q4_K_M.gguf']
Unsloth: example usage for text only LLMs: /root/.unsloth/llama.cpp/llama-cli --model short_news_gguf_v2_gguf/llama-3.2-1b-instruct.Q4_K_M.gguf
Unsloth: Saved Ollama Modelfile to short_news_gguf_v2_gguf/Modelfile
Unsloth: convert model to ollama format by running - ollama create model_name -f short_news_gguf_v2_gguf/Modelfile
total 2.4G
-rw-r--r-- 1 root root 536 Aug 19 12:19 chat_template.jinja
-rw-r--r-- 1 root root 974 Aug 19 12:19 config.json
-rw-r--r-- 1 root root 233 Aug 19 12:19 generation_config.json
-rw-r--r-- 1 root root 2.4G Aug 19 12:21 model.safetensors
-rw-r--r-- 1 root root 51K Aug 19 12:19 tokenizer_config.json
-rw-r--r-- 1 root root 17M Aug 19 12:19 tokenizer.json

```

Start coding or [generate](#) with AI.

▼ Run it in Ollama

Unsloth writes a `Modelfile` next to the GGUF with the correct Qwen3 chat template baked in. Two ways to use it:

A) On a machine with Ollama installed (your GCP L4 VM, or locally): download the `.gguf` + `Modelfile`, then:

```

ollama create pizza-palace -f Modelfile
ollama run pizza-palace

```

B) Inside Colab (to demo end-to-end):

```

# Install + start Ollama inside Colab (demo only)
!curl -fsSL https://ollama.com/install.sh | sh
import subprocess, time
subprocess.Popen(["ollama", "serve"])
time.sleep(5)

# Unsloth auto-generates a Modelfile; if you need one manually, this works too:
modelfile = '''FROM ./pizza_gguf/unsloth.Q4_K_M.gguf
TEMPLATE ""{{ if .System }}<|im_start|>system
{{ .System }}<|im_end|>
{{ end }}{{ if .Prompt }}<|im_start|>user
{{ .Prompt }}<|im_end|>
{{ end }}<|im_start|>assistant
{{ .Response }}<|im_end|>
'''

```

```
PARAMETER stop "<|im_end|>"
PARAMETER temperature 0.7
'''
open("pizza_gguf/Modelfile", "w").write(modelfile)
```